

Simple Linear Regression: Intuition and Code

Contents

1. Overview	2
2. The problem	3
2.1 Without the data: guess the average	3
3. A line through the data	3
3.1 Plotting the data	3
3.2 If the data were perfectly linear	4
3.3 Why real data is not on a line	4
3.4 The best-fit line	5
4. Linear regression in scikit-learn	6
4.1 Inputs, output and split	6
4.2 Training and predicting	6
5. What the model learned: m and b	7
5.1 Reading m and b	7
5.2 The slope m : how much the output depends on the input	7
5.3 The intercept b : the starting value	8
6. Summary	10
7. Key terms	10

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Regression problems ([Note 3](#)); Supervised learning ([Note 3](#)); Model-based learning ([Note 6](#)); Multicollinearity ([Note 27](#)).
- **Leads to:** Regression metrics ([Note 52](#)); Multiple linear regression ([Note 53](#)).

1. Overview

Key point: Linear regression fits a straight line through the data and uses that line to predict a number. Simple linear regression uses one input column.

Linear regression is usually the first ML algorithm people learn, for good reason. It is simple, easy to explain, and many later algorithms build on its ideas.

It is a **supervised** algorithm for **regression** problems, where the output column is a number, such as a price or a salary. Figure 1 shows its family.

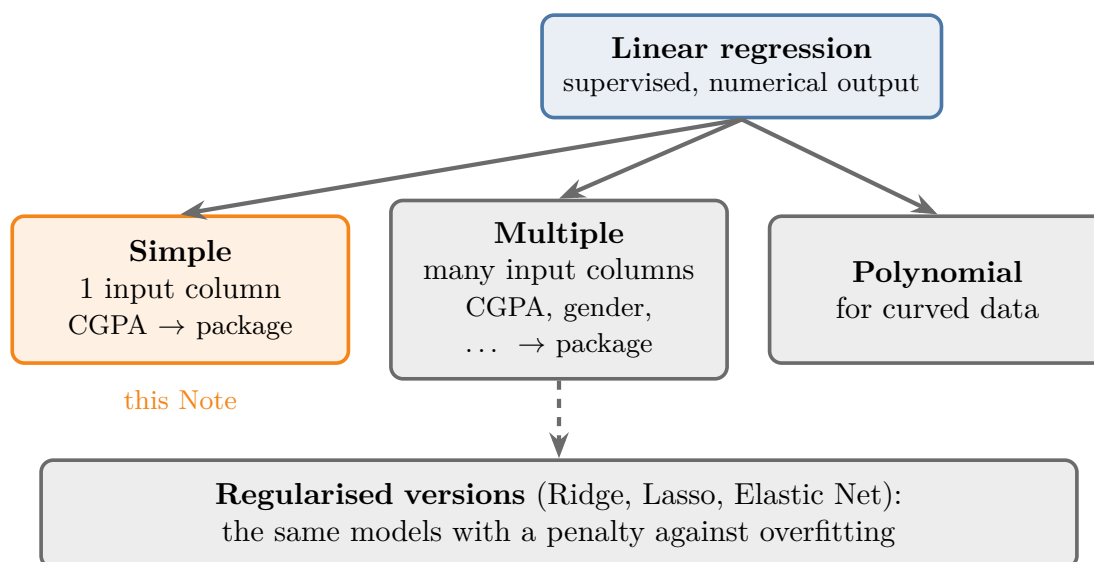


Figure 1: The linear regression family

- **Simple linear regression:** one input column and one output column. This Note.
- **Multiple linear regression:** several input columns, for example CGPA, gender and 12th-grade marks to predict a package.
- **Polynomial regression:** for data that follows a curve rather than a line.
- **Regularised versions** of these models, which add a penalty against overfitting, come later.

This Note builds the intuition and runs the scikit-learn code. The next Note derives the mathematics and writes the algorithm from scratch.

2. The problem

Key point: From a student's CGPA, predict the salary package they will be offered.

Our data describes 200 students from one college who were placed in campus recruitment. It has two columns:

cgpa	package
6.89	3.26
5.12	1.98
7.82	3.25

- **cgpa:** the student's grade point average, out of 10. This is the **input**.
- **package:** the salary offered, in **LPA** (lakh rupees per year). This is the **output**: it depends on the CGPA.

The goal is a model that takes a new student's CGPA and predicts their package. It could sit behind a small website where a student types in their CGPA.

2.1 Without the data: guess the average

Key point: With no model, the best single guess is the average package, 3.00 LPA. It is the same answer for every student, which is clearly poor.

Suppose someone asks what package a student from this college gets. Without looking at CGPA, the most reasonable answer is the average of all packages: **3.00 LPA**.

That answer ignores everything that makes one student different from another. A student with CGPA 9 and a student with CGPA 5 would get the same prediction. The red dashed line in Figure 2 is this "same guess for everyone".

3. A line through the data

Key point: The points rise roughly along a straight line. Linear regression finds the line that fits them best.

3.1 Plotting the data

Key point: Higher CGPA goes with a higher package, roughly along a straight line, with scatter around it.

The first step with a new dataset is to plot it. Figure 2 shows every student as one point.

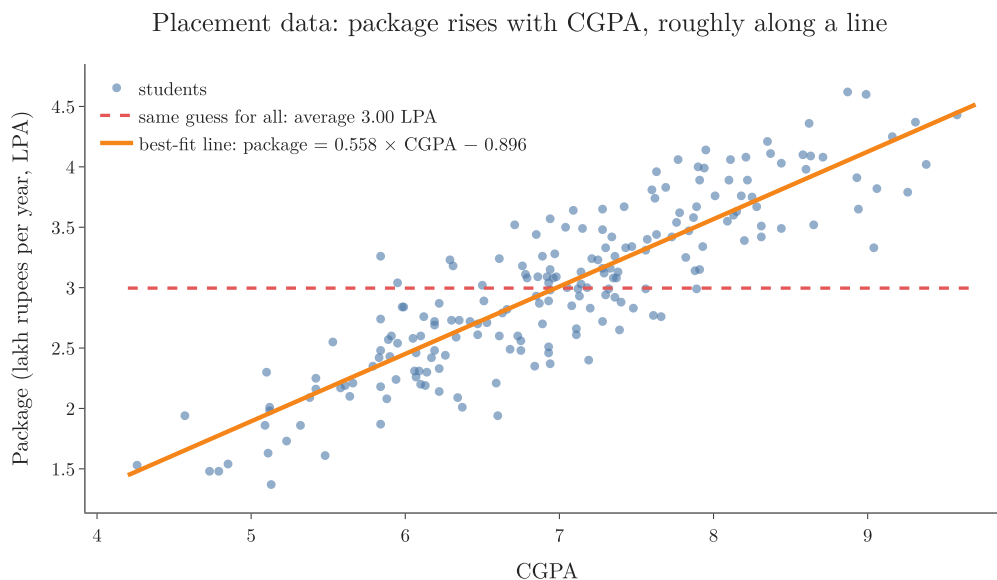


Figure 2: 200 students: CGPA and package, with the average guess and the best-fit line

The pattern is clear: students with a higher CGPA tend to get a higher package, and the points lie roughly along a straight line. Data like this is called **sort of linear**: linear in its trend, but not exactly on a line.

3.2 If the data were perfectly linear

Key point: If every point lay on one line, we would find that line's equation and use it to predict.

Imagine every student's package lay exactly on a straight line. A straight line has the equation

$$y = mx + b$$

where x is the input (CGPA), y is the output (package), m is the **slope** and b is the **intercept**. Knowing m and b , we could put any new CGPA into the equation and read off the package exactly.

3.3 Why real data is not on a line

Key point: Real data is scattered around the trend because of factors we cannot measure.

Real data is almost never perfectly linear. Two students with the same CGPA may get quite different packages:

- one may have interviewed very well, the other badly;
- one may have joined a company that pays more;

- one may have had a bad day in the test.

None of these can be captured as a column in the data. Such hard-to-measure, random influences are called **stochastic errors**, and they scatter the points around the trend.

3.4 The best-fit line

Key point: We still draw one line: the one whose total error over all points is the smallest.

Since no line can pass through every point, linear regression draws the line that is wrong by the least overall. This is the **best-fit line**.

For each student, the **error** is the vertical gap between their real package and the line's prediction. A good line keeps all these gaps small. Figure 3 compares three lines on the 160 training students.

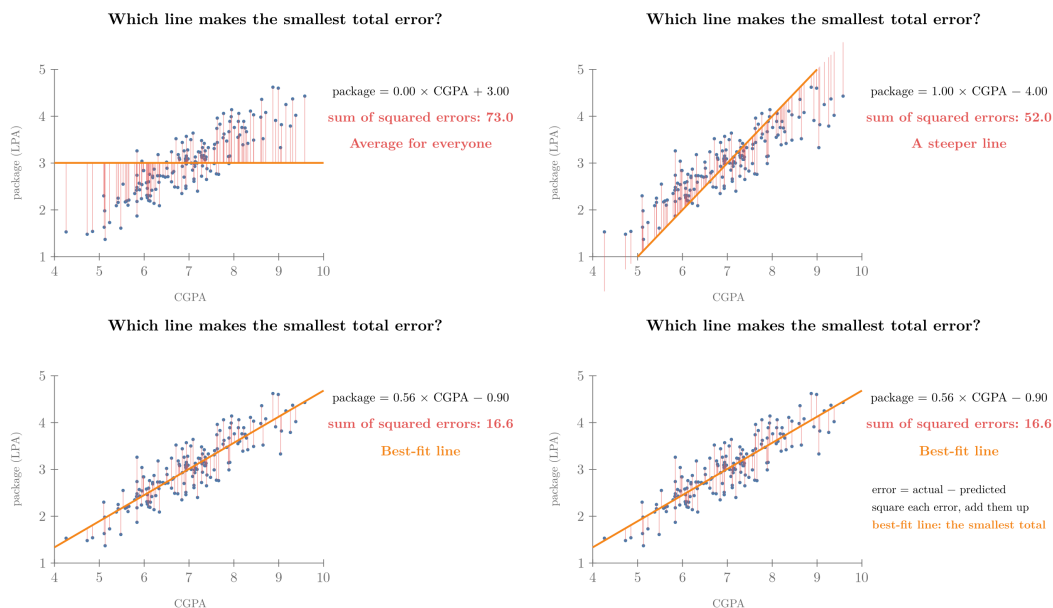


Figure 3: Three lines and their total squared error

1. **The average for everyone** (a flat line at 3.00): total squared error 73.0.
2. **A steeper line**: 52.0. Better, but it overshoots at high CGPA.
3. **The best-fit line**: 16.6, the smallest possible total.

The total used is the **sum of squared errors**: each gap is squared, so that gaps above and below the line both count, and then all are added. How the best m and b are found from this is the subject of the next Note.

4. Linear regression in scikit-learn

Key point: Split the data, create a LinearRegression object, fit it on the training set, and predict.

4.1 Inputs, output and split

Key point: X holds the CGPA column, y the package; 80% of students train the model and 20% test it.

Python: Separating the columns and splitting.

```
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
df = pd.read_csv("data/placement.csv")
X = df.iloc[:, 0:1]    # the cgpa column, kept as a table
y = df.iloc[:, -1]     # the package column
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=2)
```

160 training students, 40 test students

`iloc[:, 0:1]` keeps a DataFrame with one column, not a single column: scikit-learn expects its inputs as a 2-D table, even when there is only one input.

4.2 Training and predicting

Key point: `fit` finds the best-fit line from the training set; `predict` reads the line at a new CGPA.

Python: Training and predicting.

```
from sklearn.linear_model import LinearRegression
```

```
lr = LinearRegression()
lr.fit(X_train, y_train)    # find the best-fit line
```

```
lr.predict(X_test.iloc[[0]])    # the first test student
# array([3.89])
```

`X_test.iloc[[0]]` (double brackets) keeps the student as a one-row table. A single row with single brackets would be 1-D, and scikit-learn would refuse it.

The first test student has a CGPA of 8.58 and was offered 4.10 LPA. The model predicts 3.89 LPA. A few more test students:

CGPA	Actual (LPA)	Predicted (LPA)
8.58	4.10	3.89

CGPA	Actual (LPA)	Predicted (LPA)
7.15	3.49	3.09
5.88	2.08	2.38
6.22	2.33	2.57
4.57	1.94	1.65

The predictions are close, but not exact: the stochastic errors of Section 3.3 cannot be predicted from CGPA. How to measure a regression model's accuracy properly is the subject of a later Note on regression metrics.

5. What the model learned: m and b

Key point: The trained model is just two numbers: the slope $m = 0.558$ and the intercept $b = -0.896$.

5.1 Reading m and b

Key point: `coef_` holds the slope and `intercept_` holds the intercept.

The best-fit line is fully described by its slope and intercept. scikit-learn stores them after fit.

Python: The line's two numbers.

```
m = lr.coef_[0]          # 0.558
b = lr.intercept_        # -0.896
```

So the model is the equation

$$\text{package} = 0.558 \times \text{CGPA} - 0.896$$

With numbers, for the student with CGPA 8.58:

$$0.558 \times 8.58 - 0.896 = 4.788 - 0.896 = 3.89$$

This is exactly the prediction from `lr.predict`. All `predict` does is put the CGPA into this equation.

5.2 The slope m : how much the output depends on the input

Key point: Each extra CGPA point adds 0.56 LPA to the predicted package.



Figure 4: The slope and the intercept of the best-fit line

The slope is the change in the output for one unit of change in the input (the green triangle in Figure 4). Here, every extra CGPA point adds 0.56 LPA to the predicted package.

So m acts like a **weight**: it says how strongly the output depends on the input.

- A slope near 0 would mean the package hardly depends on CGPA.
- A large slope would mean a small change in CGPA moves the package a lot.

5.3 The intercept b : the starting value

Key point: The intercept is the line's value when the input is 0. Sometimes it has a real meaning, sometimes it does not.

The intercept is where the line crosses the vertical axis: the prediction for an input of 0. Here, $b = -0.90$ LPA at CGPA 0 (the red point in Figure 4).

That value has no real meaning: no student has a CGPA of 0, and a negative salary is impossible. It is just where the line has to start so that it fits the students we do have, between CGPA 4 and 10.

In other data, the intercept can mean something real.

Take years of experience and salary: a fresher has 0 years of experience but still earns a salary. There, the intercept is the starting salary, and it should not be 0. The intercept is what lets the line start at the right height.

Extra: The line does not know where the real data ends. For CGPA 10 it predicts 4.68 LPA, which is reasonable. For CGPA 100, which cannot exist, it calmly predicts 54.9 LPA. A model is only trustworthy within the range of the data it was trained on;

predicting outside that range is called **extrapolation**.

6. Summary

Idea	On the placement data
Input x and output y	CGPA; package in LPA
Guess without a model	the average, 3.00 LPA for everyone
Best-fit line	$\text{package} = 0.558 \times \text{CGPA} - 0.896$
Slope m	+0.56 LPA per CGPA point
Intercept b	-0.90, meaningless here (no CGPA of 0)
Total squared error, training set	73.0 for the average; 16.6 for the best-fit line

- Linear regression is supervised and predicts a number; simple linear regression uses one input.
- Real data is “sort of linear”: a linear trend plus scatter from stochastic errors.
- The best-fit line is the line with the smallest sum of squared errors.
- In scikit-learn: `LinearRegression().fit(X_train, y_train)`, then `predict`.
- The trained model is two numbers: `coef_` (slope, the input’s weight) and `intercept_` (the starting value).
- The line keeps going beyond the data; predictions far outside the training range are unreliable.

7. Key terms

Term	Meaning
Linear regression	A supervised algorithm that predicts a number with a straight line (or flat surface) through the data
Simple linear regression	Linear regression with one input column
Multiple linear regression	Linear regression with several input columns
LPA	Lakh rupees per annum: a salary in hundreds of thousands of rupees per year
Slope	How much the output changes for one unit of change in the input; m in $y = mx + b$
Intercept	The line’s value when the input is 0; b in $y = mx + b$
Stochastic error	A random, unmeasurable influence that scatters data around its trend
Best-fit line	The line with the smallest total error over all the training points
Error (residual)	The gap between an actual value and the model’s prediction
Sum of squared errors	The squares of all the errors added up; the quantity the best-fit line makes smallest

Term	Meaning
coef_	The fitted slope (one per input column) in scikit-learn
intercept_	The fitted intercept in scikit-learn
Extrapolation	Predicting for inputs outside the range of the training data